oops :-

* class & classroom ✓

}

:: → scope resolution operator

* access specifier:-
  - ↳ public
  - ↳ protected
  - ↳ private

* base class = parent class
* derived class = child class

* Instance of class = object (int main() {
  classroom obj;
  }

* data members ✓
  ↳ ~~functions~~ variables inside class.

* data functions
  ↳ function/methods inside class

* Constructor:- ✓ (runs automatically)

```
class hi {
    hi(){...}   <- runs automatically
    or
    hi(){...};
}
```

* destructor:-

```
class hi {
    hi(){};

    ~hi();
}
```

* function overloading
  ↳ using same name of function with different datatypes

(layman)

* static = constant for every object inside a class

* Virtual = If any problem occurs due to ambiguity, then this function will get ignored.

* override = use to redefine any function for a specific child.

* Abstract ⇒ khali template / blueprint

≥ Inheritance → inheriting properties (variables) and behaviour (functions) from parent class to child class

① single inheritance:-

```
class A {
public:
void display() { ... };
};
class B : public A {
// it will inherit all things of A
};
```

② multiple inheritance

```
class C : public A, public P {
// inherit both A, P.
};
```

```
class A {
public: void display1() { };
};
class P {
public: void display2() { };
};
```

Note:- if A, P contains same method name, then this code will show error ⇒ ambiguous error (red line)

* class C : public A, virtual public P {
// inherits display of A
};

* class P : public P, virtual public A {
// inherits display of P
};

```
class A {
void display() { };
};
class P {
void display() { };
};
```

③ ~~hierarchial~~ multilevel inheritance

A
↓ child
B
↓ child
C

* child ~ : ~ {
// inherits ✓
};

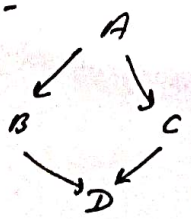
here we can write extra code for this child. other than what it has inherited.

④ ~~multilevel~~ hirorchial inheritance :-

A
child ↙ ↘ child
B C

⑤ hybrid inheritance :-

multiple + hierarchial




```
class A {
public: void show() { };
};
class B : virtual public A { }; // extras
class C : virtual public A { }; // extras
class D : public B, public C { }
// here D will only inherit A once.
```

⑦ function overiding :-

```
class Base {
public:
  virtual void display() { ---- };

};

class derived : public Base {

      void display() { ---- };

};
```

we have over-rided the Base's display now this display will execute

⑧ Abstract class :- and pure virtual function

```
class Abstract {
  public:
    virtual void display() = 0;

};

class derived: public Abstract {
    public:
      void display() { ---- }

};
```

here we used pure virtual function therefore its called Abstract class.

over-rided the pure virtual function.

⑨ Constants data member and Constant member function :-

must be called in Constructor

~~can be called without creating instance of class~~

```
class Example {
    const int x;   ← constant data member

public:
    Example (int value) : x (value) { };
    void show () const { ---- };   ← constant member function
};
```

⑩ static data member and static member function

↳ can be shared across all objects of class

↳ can only work on static data member. can't on class objects

↳ can be called without creating instance of class.

```
class example {
    static int count;
public:
    static int getcount() {
        return count;
    };
```

```
class myclass {
  public:
    static int display() { };
};
int main() {
    myclass :: display();
    return 0;
}
```

⊙ polymorphism:-

- Compile Time polymorphism → achieved using function overloading or operator overloading
- Run-time polymorphism → achieved by virtual function of base class overrided by its child class.

• Virtual function = declared in Base class to enable runtime polymorphism

⊙ function overloading :-

```
class addition {
public:
    int add (int a, int b) {
        return a+b;
    }
    float add (float a, float b) {
        return a+b;
    }
    int add (int a, int b, int c) {
        return a+b+c;
    }
};
```

⊙ operator overloading :-

```
class complex {
    int real, imag;
public:
    complex (int r, int i) : real(r), imag(i) {};
    complex operator+ (const complex& obj) {
        return complex (real + obj.real, imag + obj.imag);
    }
};
int main() {
    complex c1, c2;
    complex c3 c1+c2;
}
```

output ⇒

⊙ Dynamic Binding → Resolves method calls at runtime using virtual functions.

⊙ use of virtual → in overriding
→ in Dynamic Binding

Virtually Inherited from P | Virtually Inherited from P

A, B → C (it will only have P once)

Exception handling :-

```
try {
    if (a==0) {
        throw "error occured";
    }
}
catch (const char *msg) {
    cout << msg << endl;
}
```

① Templates :-

```
template <typename T>
```
(or int, void, char, class etc.)

```
T add (T a, T b) {
    Return a+b;
}
```
↘ here T will automatically become int for everywhere.

② Stream classes → iostream (for console)
→ fstream (for files)

⋮

⋮